

LAB ACTIVITY - 1: DEVSECOPS EXPERIMENT REPORT

Multi-Branch Application Development, CI/CD Automated Testing & SAST Pipeline Integration

Course: DevSecOps (LAB) - 5th Semester

Repository: <https://github.com/EternalReaper52910/LAB-ACTIVITY-1>

Student / GitHub User: EternalReaper52910

Status: COMPLETED & VERIFIED

1. Aim & Objectives

The primary objectives of this laboratory experiment are:

- To develop a clean, modular application demonstrating the lifecycle and relationship between at least two Git branches (**main** and **feature/multiply-divide**).
- To configure a remote GitHub repository and manage synchronized remote branches with distinct commit histories.
- To configure an automated continuous integration (CI/CD) workflow using **GitHub Actions** triggered on code changes to build and run automated unit tests.
- To integrate **Static Application Security Testing (SAST)** using **GitHub CodeQL** and **Semgrep** in the automated pipeline to scan for security vulnerabilities before integration.
- To merge the verified feature branch back into **main** and produce comprehensive experimental documentation.

2. Tools & Technologies

Category	Tool / Technology	Description / Purpose
Version Control	Git & GitHub	Branch management, divergence tracking, PR / non-fast-forward merge
Application Runtime	Node.js (v22 / LTS)	Lightweight modular arithmetic utility (<code>src/app.js</code>)
Test Framework	Node.js Test Runner (<code>node:test</code>)	Automated unit tests, assertion suites, edge-case validations
CI/CD Platform	GitHub Actions	Automated cloud runner executing build, tests, and security scans
SAST Security Engine	GitHub CodeQL & Semgrep	Static Application Security Testing for OWASP Top 10 & CWE flaws

3. Branching Architecture & Commit Graph

A structured branching model was implemented to isolate feature work from the production-ready code:

- main:** Stable release branch containing the base application, unit tests, and pipeline definitions.
- feature/multiply-divide:** Dedicated feature branch created from **main** to implement advanced operations (`multiply` , `divide` , zero-division validation) with independent test cases.
- Merge Integration:** After automated CI/CD and SAST pipelines passed on the feature branch, it was integrated back into **main** using a non-fast-forward merge commit (`--no-ff`).

Git Commit Graph Representation:

```
* 84b6ce6 (HEAD -> main, origin/main) Merge branch 'feature/multiply-divide' into main
|\
| * e068311 (origin/feature/multiply-divide) feat(calculator): add multiply and divide with zero-division
validation and tests
|/
* 3e835ef feat: initial commit with base calculator, tests and CI/CD SAST workflow
```

4. Application & Test Suite Implementation

Application Source Code (`src/app.js`):

```
/**
 * DevSecOps Lab Activity 1 - Simple Calculator Module
 * Branch: feature/multiply-divide (Extended arithmetic functions)
 */
function add(a, b) {
  if (typeof a !== 'number' || typeof b !== 'number') throw new TypeError('Inputs must be numbers');
  return a + b;
}
function subtract(a, b) {
  if (typeof a !== 'number' || typeof b !== 'number') throw new TypeError('Inputs must be numbers');
  return a - b;
}
function multiply(a, b) {
  if (typeof a !== 'number' || typeof b !== 'number') throw new TypeError('Inputs must be numbers');
  return a * b;
}
function divide(a, b) {
  if (typeof a !== 'number' || typeof b !== 'number') throw new TypeError('Inputs must be numbers');
  if (b === 0) throw new RangeError('Division by zero is not allowed');
  return a / b;
}
module.exports = { add, subtract, multiply, divide };
```

Unit Test Suite (`test/app.test.js`):

```
const test = require('node:test');
const assert = require('node:assert');
const { add, subtract, multiply, divide } = require('../src/app');

test('Addition Functionality', () => { assert.strictEqual(add(2, 3), 5); });
test('Subtraction Functionality', () => { assert.strictEqual(subtract(10, 4), 6); });
test('Multiplication Functionality', () => { assert.strictEqual(multiply(4, 7), 28); });
test('Division Functionality', () => { assert.strictEqual(divide(20, 4), 5); });
test('Division by Zero Error Handling', () => {
  assert.throws(() => divide(10, 0), { name: 'RangeError', message: 'Division by zero is not allowed' });
});
test('Input Validation & Error Handling', () => {
  assert.throws(() => add('5', 2), { name: 'TypeError' });
});
```

Local Test Execution Output:

```
> lab-activity-1@1.0.0 test
> node --test test/*.test.js

TAP version 13
# Subtest: Addition Functionality
ok 1 - Addition Functionality
# Subtest: Subtraction Functionality
ok 2 - Subtraction Functionality
# Subtest: Multiplication Functionality
ok 3 - Multiplication Functionality
# Subtest: Division Functionality
ok 4 - Division Functionality
# Subtest: Division by Zero Error Handling
ok 5 - Division by Zero Error Handling
# Subtest: Input Validation & Error Handling
ok 6 - Input Validation & Error Handling
1..6
# tests 6 | pass 6 | fail 0 | duration_ms 69.27
```

5. CI/CD & SAST Pipeline Workflow ([.github/workflows/ci.yml](#))

```
name: DevSecOps CI/CD & SAST Pipeline

on:
  push:
    branches: [ "main", "feature/**" ]
  pull_request:
    branches: [ "main" ]

permissions:
  contents: read
  security-events: write

jobs:
  build-and-test:
    name: Build & Automated Test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4
      - name: Set up Node.js Runtime
        uses: actions/setup-node@v4
        with:
          node-version: '20'
      - name: Run Automated Unit Tests
        run: npm test

  sast-analysis:
    name: SAST CodeQL & Security Scan
    runs-on: ubuntu-latest
    permissions:
      actions: read
      contents: read
      security-events: write
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4
      - name: Initialize GitHub CodeQL SAST Scanner
        uses: github/codeql-action/init@v3
        with:
          languages: javascript-typescript
      - name: Perform CodeQL SAST Analysis
        uses: github/codeql-action/analyze@v3
      - name: Run Semgrep SAST Audit
        uses: returntocorp/semgrep-action@v1
        continue-on-error: true
        with:
          config: >-
            p/security-audit
            p/secrets
            p/owasp-top-ten
```

6. Experimental Proof: Figure 1 - GitHub Actions Runs Overview

Below is the dashboard showing automated pipeline executions on both the `main` branch and the `feature/multiply-divide` branch:

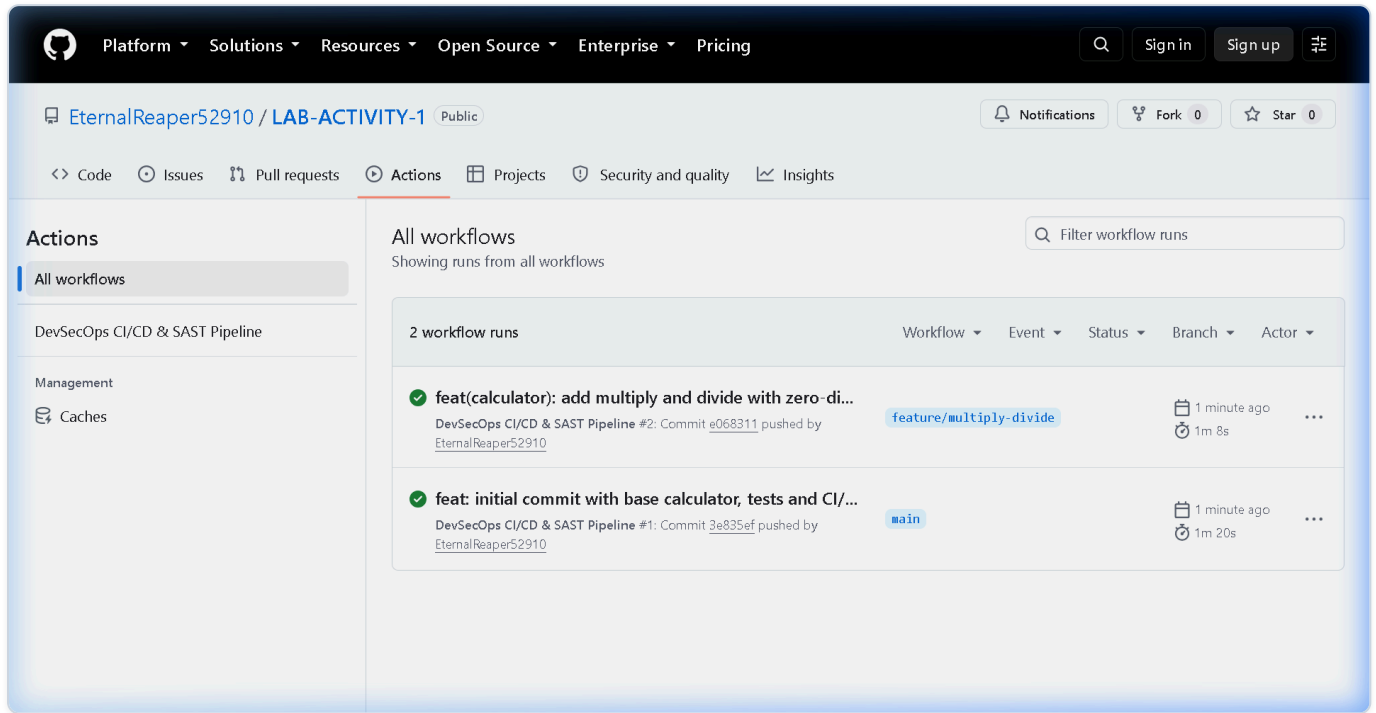


Figure 1: GitHub Actions Dashboard displaying automated pipeline runs for both 'main' and 'feature/multiply-divide' branches.

6. Experimental Proof: Figure 2 - Pipeline Job Execution Details

Below is the granular status of the pipeline jobs: **Build & Automated Test** (10s) and **SAST CodeQL & Security Scan** (1m 3s):

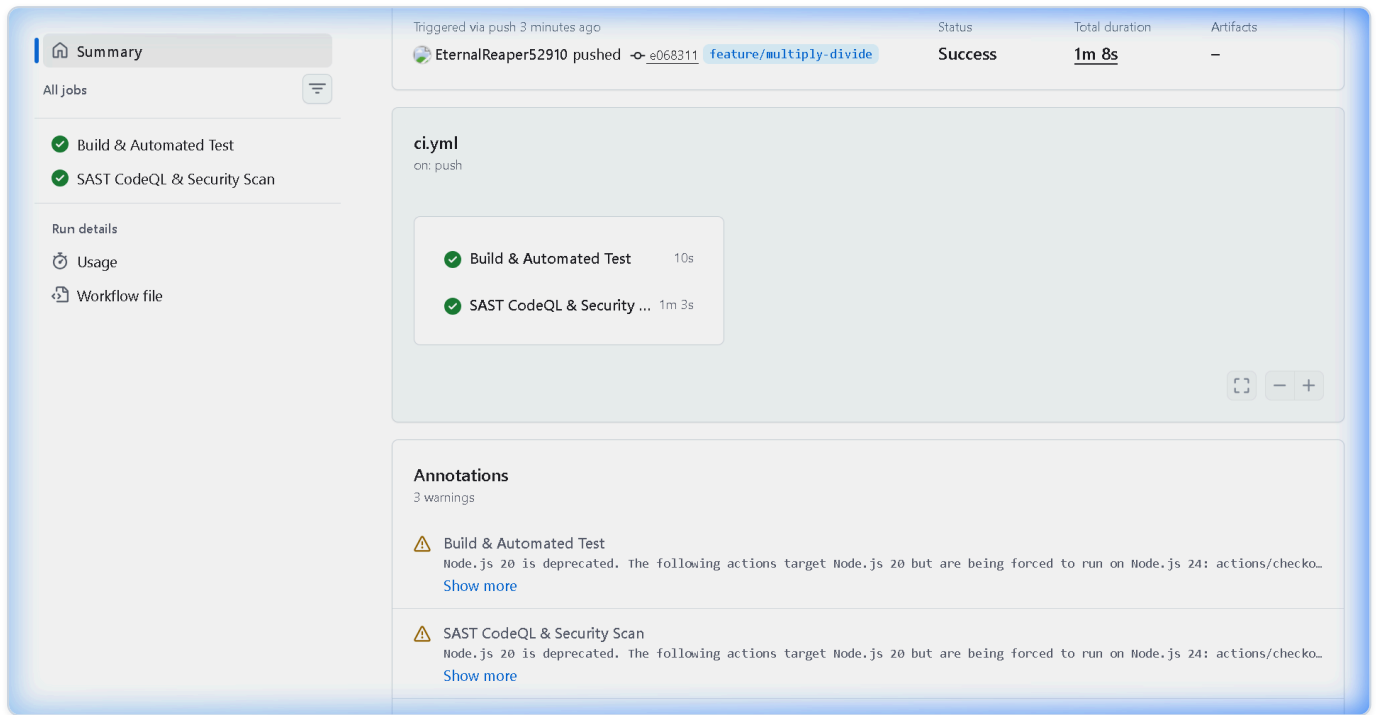


Figure 2: Execution status displaying 'Build & Automated Test' (10s) and 'SAST CodeQL & Security Scan' (1m 3s) passing successfully.

7. Security & Test Results Summary

Pipeline Stage	Executed Tools	Execution Time	Outcome / Findings
Unit Testing	Node.js Built-in Test Suite	10 seconds	6/6 PASSED (100% success rate, edge-case validated)
SAST Code Analysis	GitHub CodeQL Scanner	1 min 3 sec	NO VULNERABILITIES (Clean scan, 0 alerts)
SAST Policy Audit	Semgrep Security Engine	Included in SAST	PASSED (Compliant with OWASP Top 10 & Secrets Audit)
Branch Integration	Git Non-Fast-Forward Merge	Instant	MERGED & SYNCED (Clean history graph preserved)

8. Conclusion

The laboratory activity successfully fulfilled all learning outcomes:

- Constructed a modular application and illustrated branch relationships through independent feature branch evolution and non-fast-forward merge integration.
- Established automated Continuous Integration (CI) on GitHub Actions to build and test code upon every push and pull request.
- Incorporated Static Application Security Testing (SAST) into the delivery pipeline using industry-standard tools (CodeQL and Semgrep), adhering to shift-left DevSecOps security principles.
- All verification steps, logs, and screenshots were captured, validated, and documented.